



软件架构模式

吕骏
南京大学





个人简介



教育&工作经历



吕骏，目前在南京大学软件学院担任助理研究员
研究方向涵盖软件测试、软件安全及AI4SE（人工智能赋能软件工程）等领域过去的研究主要关注软件构建过程的有效性与效率；目前重点探索将大语言模型（LLMs）集成于软件研发全流程以及软件构建安全性在本领域CCF-A类会议与期刊（如ICSE、FSE、ASE、ISSTA、TSE、和TOSEM等）发表论文十余篇，申请国家发明专利多项，并参与国家自然科学基金项目同时，与新加坡国立大学、新加坡管理大学多位教授保持长期科研合作

 <https://meiye-lj.github.io/>

 junlyu@nju.edu.cn

 025-83621360-405



绪论



为什么要把架构放回时代里讲

如果不理解“旧模式为何失效”，就很难理解“新模式为何成立”

架构不是凭空发明的

每一种架构，都在回应当时最紧迫的技术与业务矛盾

真正要学的是迁移逻辑

知道旧模式为什么失效，才知道新模式为什么出现

“先进”是相对的

同一种模式在一个时代是解药，在另一个时代可能就是负担

AI 时代尤为如此

AI 原生不是“微服务 + LLM”，而是主要矛盾再次迁移后的新骨架

这门课最终要训练的不是“记住术语”，而是“看懂时代矛盾并做边界取舍”



时代的主线：从算力稀缺到机器认知

一条连续主线：谁是系统的核心约束，谁就会重画边界



架构演进不是“替代史”，而是“主要矛盾迁移史”

算力稀缺 → 多用户共享 → 富交互 → 系统内分层 → 企业级协同 → 团队自治发布 → 平台化韧性
→ 语义增强 → 认知式执行

所以：不是越新越好，而是要判断你的系统今天到底卡在哪一层矛盾



从“质量属性”看架构

同一种架构，不是绝对先进，而是在当时约束下重排质量属性优先级

架构选择的本质，是在特定时代约束下，对质量属性做“有意识的优先级排序”

质量属性不会同时最优

性能、可维护性、可扩展性、安全性、可部署性等常常彼此拉扯。

时代变化会重排优先级

约束会从算力、组织、流量一路迁移到语义与认知，取舍也会跟着迁移。

架构选择本质是排序

谁是当时主矛盾，谁就决定系统优先保什么、允许牺牲什么。

所以后面每章都看一件事

当时为了缓解主矛盾，系统在质量属性上到底做了什么边界取舍。



从一个熟悉的问题开始：选课系统为什么会不断重构？

切入点

假设学校要建设一套“课程与学习服务系统”：学生选课、退课、查课表；老师录成绩、看名单；教务员排课、控容量、处理冲突。

案例

它同时包含事务一致性、交互体验、跨系统集成、高峰流量、通知协同、知识问答和 AI 规划。

学生

老师

教务员

学校平台

课程 / 名单 / 成绩 / 权限 / 培养方案

核心追问

如果这个系统从 1960s 一直演进到 AI 时代，它的架构会怎样变化？为什么必须变化？



最早期软件开发

1940s–1960s

软件首先是“让机器稳定执行任务”的方法，而不是独立产品





时代背景：技术条件、业务需求与默认假设

1940s–1960s

这一个时代在发生什么？

- 硬件极其昂贵，编程首先服务于“让机器跑起来”
- 应用以科学计算、军工计算、批处理报表为主，交互几乎不存在
- 软件与硬件、操作员流程、具体任务强绑定，复用性很弱

技术关键词

真空管/晶体管计算机

汇编与早期高级语言

这一阶段的默认架构目标

先让机器稳定运行

提高昂贵算力利用率

把重复计算做成制度化流程

默认目标会决定：当代架构优先优化什么，又会忽略什么

最早期软件开发

主机/终端

C/S 架构

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



主要矛盾：这一代架构真正想解决什么

主要矛盾：计算资源极其稀缺，但任务正在从“算一次”走向“重复算、稳定算、规模算”

资源

CPU/内存/存储昂贵，首先追求机器利用率

需求

任务固定，但结果必须稳定、可重复

工程

程序往往为一台机器、一项任务定制

组织

程序员、操作员、业务之间隔着很长链路

理解“主要矛盾”比记住术语更重要：它决定了那一代架构为什么看起来“理所当然”

最早期软件开发

主机/终端

C/S 架构

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



最早期软件开发的结构骨架：边界、职责与协作

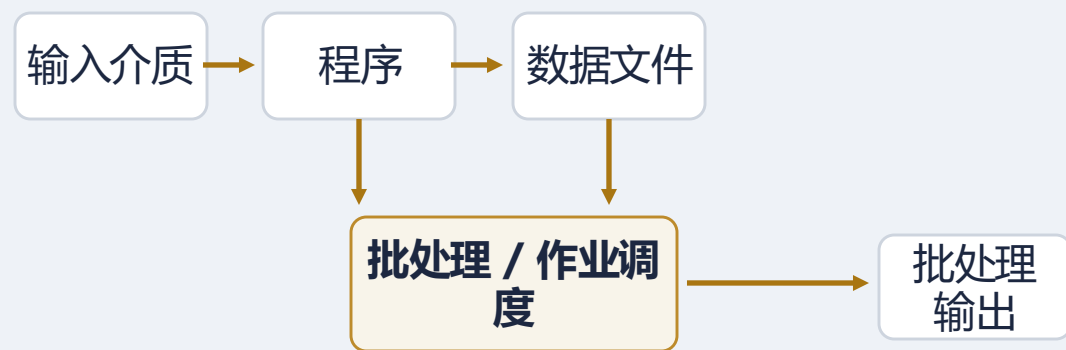
1940s–1960s

结构抓手

- 程序与数据组成作业，统一送入系统执行
- 执行围绕批处理与调度组织，强调吞吐与稳定
- 交互极弱，输出通常在作业完成后统一返回

本质：先把“重复任务”写进程序，再把“执行秩序”写进流程

结构骨架





主机 / 终端



这一代为什么会出现：新的约束与核心矛盾

1960s–1980s

新约束：企业事务数字化；核心矛盾：越来越多用户需要共享同一套核心数据与事务能力

技术条件

大型主机成为中心计算平台，终端负责输入与显示

默认目标

保证中心事务正确、安全，并让多终端共享访问

业务需求

银行、票务、工资、账务等重复事务开始在线化

暂时忽略

丰富交互与部门级灵活定制仍让位于集中治理

这一代第一次真正把“共享核心系统”做成了企业计算的常态

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

1960s–1980s

上一代模式：单机 / 批处理

- 擅长固定任务的稳定执行
- 强调昂贵算力利用率与结果可复现
- 对离线作业和报表很有效

新约束下的问题

- 无法支撑大量用户持续在线访问同一数据
- 缺少统一的事务、权限与审计控制
- 数据共享与并发访问一多，批处理思路就会失灵

因此，系统必须从“执行任务”转向“服务一群用户”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



主机 / 终端的结构骨架：边界、职责与协作

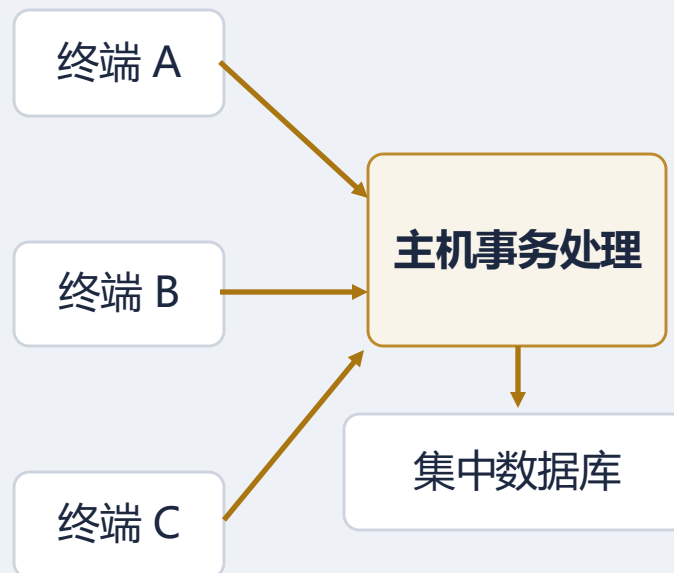
1960s–1980s

结构抓手

- 主机承担几乎全部事务处理、数据存储与权限控制
- 终端主要负责输入、显示，不承载复杂业务逻辑
- 安全、审计和一致性都被集中到中心系统

本质：通过“强中心 + 弱终端”换取一致性、安全性与治理能力

结构骨架





主机 / 终端如何解决核心问题

1960s–1980s

它通过事务中心化，把“多人共享”从混乱访问变成可治理的制度化系统

1 统一事务

同一份核心数据由中心系统负责一致性

2 统一安全

权限、审计与访问路径集中控制

3 统一资源共享

昂贵算力服务更多终端用户

4 统一运维治理

关键系统更容易被制度化管理

解决的是“很多人如何共享核心事务系统”；但交互灵活性依然很弱

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



主机 / 终端的质量属性取舍

1960s–1980s

通过“强中心 + 弱终端”，把事务正确与集中治理做到最强

核心矛盾

越来越多用户要共享同一套核心事务与核心数据。

优先保障

一致性、安全性、可靠性、可审计性。

相对牺牲

易用性、交互响应性、局部可修改性。

一句话取舍

中心化让事务和治理最稳，但也把交互贫瘠与变化成本一并锁进了中心。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



回到选课系统：主机 / 终端时代

1960s–1980s



1

核心变化

从批处理名单到多终端
共享教务数据

2

为什么这样设计

保证成绩、选课、容量
一致

3

新问题

交互弱，变化都压在中心

这一代的选课系统，本质是在用“强中心 + 弱终端”换一致性与治理能力。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



C / S 架构

1980s–1990s

PC 革命把交互能力前移，软件第一次大规模贴近桌面用户





这一代为什么会出现：新的约束与核心矛盾

1980s–1990s

新约束：PC 与 GUI 普及；核心矛盾：用户开始要求更丰富、更快速的桌面交互

技术条件

PC、Windows、LAN 普及，客户端开始具备本地算力

业务需求

软件深入财务、办公、制造、销售等桌面场景

默认目标

把交互带到桌面端，并利用本地算力改善体验

暂时忽略

统一部署、版本治理与数据边界还没有成为首要问题

这一代第一次把“用户体验”作为企业软件可感知的目标

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

1980s–1990s

上一代模式：主机 / 终端

- 把事务与数据集中管理得非常好
- 强审计、强安全、强一致性
- 适合关键共享业务

新约束下的问题

- 终端太“瘦”，难以支持复杂 GUI 与本地交互
- 所有变化都压到中心系统，部门级快速建设很难
- PC 算力已可用，完全不下放逻辑变得不经济

因此，系统开始把“交互”前移到客户端，把“共享数据”留在服务器

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



C / S 架构的结构骨架：边界、职责与协作

1980s–1990s

结构抓手

- 客户端负责 UI 与部分本地逻辑，提升响应性与体验
- 服务器负责共享数据与部分公共逻辑
- 前后端通过网络协作，形成分布式桌面计算

本质：把“富交互”从中心系统前移到桌面端

结构骨架

客户端
UI + 部分逻辑

应用服务器

数据库

胖客户端把交互做强，但也把部署复杂度带到了桌面端



C / S 架构如何解决核心问题

1980s–1990s

它通过“客户端做交互、服务器保数据”的分工，第一次把企业软件真正贴近了桌面用户

1

界面前移

GUI 与本地状态处理让体验显著改善

2

响应更快

一部分逻辑在本地完成，不必事事回中心

3

建设更快

部门级系统能较快上线，贴合局域网办公

4

共享仍存在

核心数据依旧保留在服务器端统一管理

解决的是“怎样做富交互”；但它把部署和版本复杂度也搬到了客户端

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



C/S 架构的质量属性取舍

1980s–1990s

C/S 把交互能力前移到桌面端，第一次显著提升了企业软件的用户体验

核心矛盾

用户开始要求更丰富、更即时的桌面交互。

优先保障

易用性、响应性、交互体验。

相对牺牲

可部署性、可维护性、兼容性、统一治理。

一句话取舍

它用胖客户端换来了更好的体验，但也把安装、升级和版本控制问题带到了桌面端。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



回到选课系统：C / S 时代

1980s–1990s



交互能力下放到桌面端，核心课程与名单仍留在服务器端。

这一代的选课系统，用胖客户端换来了更好的桌面体验，也带来了版本治理负担。

1

核心变化

交互从中心系统前移到桌面客户端

2

为什么这样设计

响应更快，界面更丰富

3

新问题

安装、升级、兼容成本上升

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

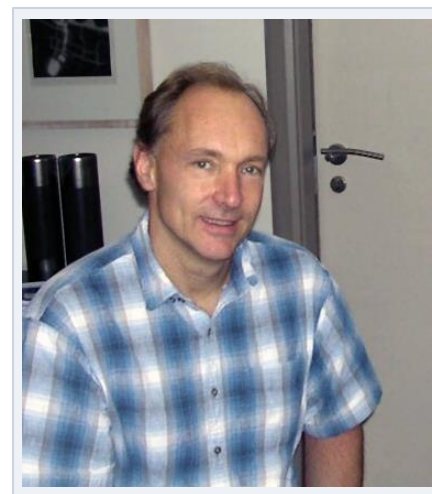
AI 原生



三层 / 分层

1990s–2000s

浏览器统一入口后，系统开始强调“服务器端分责”





这一代为什么会出现：新的约束与核心矛盾

1990s–2000s

新约束：Web 成为统一入口；核心矛盾：要降低部署复杂度，并把系统内部职责整理清楚

技术条件

浏览器普及，Web / App Server 成为企业软件主流承载方式

业务需求

用户数扩大，统一入口与统一升级变得更加重要

默认目标

瘦客户端接入，服务端集中逻辑，内部按职责清晰分层

暂时忽略

系统之间如何协同，尚未成为最尖锐的问题

这一代真正的进步，不是“能上网了”，而是“系统内部开始更易维护”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

1990s–2000s

上一代模式：C / S

- 桌面交互丰富，响应性强
- 很适合局域网与部门级应用
- 能快速把软件贴近终端用户

新约束下的问题

- 客户端安装、升级、兼容成本随着规模上升而爆炸
- 客户端直接或强耦合访问数据层，不利于统一治理
- 一个系统里混杂了太多展示、业务和数据处理

因此，重心从“前后分工”进一步转向“服务端内部清晰分层”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



三层 / 分层的结构骨架：边界、职责与协作

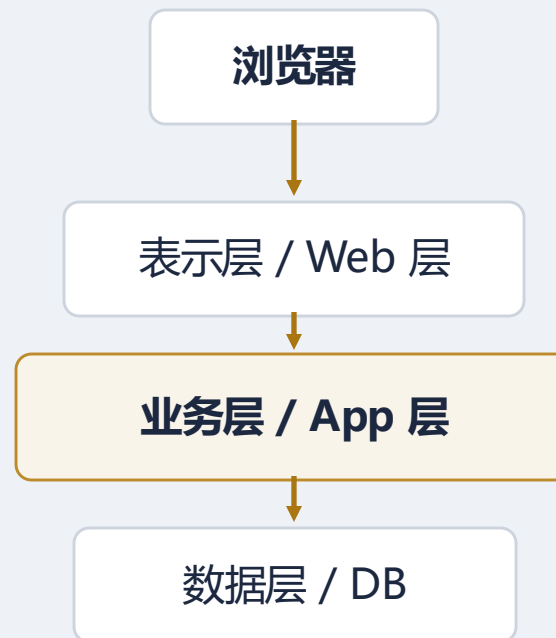
1990s–2000s

结构抓手

- 表示层负责接入与界面呈现，统一用户入口
- 业务层承载业务规则、流程控制与领域逻辑
- 数据层负责持久化与数据访问，形成清晰职责边界

本质：通过职责分层控制变化影响，而不是只做“客户端瘦身”

结构骨架



最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



三层 / 分层如何解决核心问题

1990s–2000s

它通过统一入口与职责分层，把“部署问题”和“维护问题”同时压了下来

1

浏览器统一接入

客户端安装与升级成本明显下降

2

逻辑集中到服务端

业务规则更容易统一修改与治理

3

分层控制变化

界面变化、业务变化、数据变化尽量局部化

4

安全更清晰

数据访问不必暴露到大量客户端

解决的是“系统内如何更清楚”；但并没有解决“系统间如何协同”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



三层 / 分层的质量属性取舍

1990s–2000s

分层的关键价值不是“能上网”，而是把单系统内部职责理顺、把变化影响局部化

核心矛盾

既要统一入口，又要降低维护成本，并把系统内部职责整理清楚。

优先保障

可维护性、可修改性、安全性、可部署性。

相对牺牲

极致性能、跨系统协同灵活性、快速局部自治。

一句话取舍

分层特别适合单系统治理，但复杂度仍然持续堆积在单应用边界之内。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

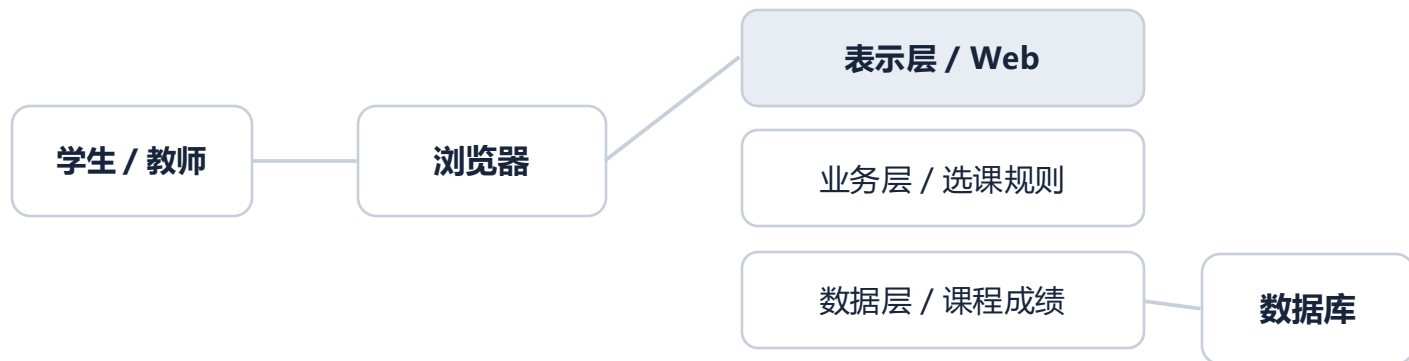
AI 原生



回到选课系统：三层 / 分层时代

1990s–2000s

服务端分层



1

核心变化

浏览器统一入口，服务端分层

2

为什么这样设计

降低客户端维护成本

3

新问题

复杂度仍堆在单应用内部

重点从“桌面交互”转向“统一入口 + 系统内职责分清”。

这一代的选课系统，把重点从“桌面交互”转向“统一入口 + 系统内分责”。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



SOA

2000s

企业第一次系统性地把“能力复用”和“流程贯通”放到架构中心



这一代为什么会出现：新的约束与核心矛盾

2000s

新约束：系统数量激增；核心矛盾：企业需要跨系统复用业务能力，并把端到端流程打通

技术条件

ERP、CRM、SCM、门户、支付等系统并存，异构成为常态

默认目标

把系统功能提升为服务能力，通过契约与治理保持秩序

业务需求

同一条业务流程开始跨部门、跨系统、跨组织运行

暂时忽略

快速独立发布与团队自治还没有压过企业级整合诉求

问题边界从“一个系统如何组织”转向“很多系统怎样协作”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

2000s

上一代模式：三层 / 分层

- 很擅长把单个系统内部职责划清
- 统一入口与服务端治理都更容易
- 对单应用维护性帮助很大

新约束下的问题

- 系统之间依旧割裂，集成常靠大量点对点接口
- 同一业务能力在不同系统中重复实现
- 跨系统流程一变动，协调成本就会迅速上升

因此，企业需要从“应用视角”上升到“服务能力视角”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



SOA 的结构骨架：边界、职责与协作

2000s

结构抓手

- 能力通过服务契约暴露，不再只藏在某个应用内部
- ESB / 中间件负责路由、转换、编排与集成
- 治理体系负责版本、规范、注册、权限与生命周期

本质：把“系统功能”提升为“可复用的服务能力”

结构骨架





SOA 如何解决核心问题

2000s

它通过“**契约 + 总线 + 编排 + 治理**”，把企业从孤岛系统集成拉向了能力网络

1

契约标准化

能力接口与系统内部实现脱钩

2

服务复用

订单、客户、支付等能力不必每个系统重写

3

流程编排

跨系统业务流程可以显式串起来

4

异构整合

不同技术栈也能进入统一企业流程

解决的是“**全企业协同**”；代价是治理体系更重、中心中间件更强

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



SOA 的质量属性取舍

2000s

SOA 优先解决“企业级协同”，所以愿意接受更重的契约、总线和治理成本

核心矛盾

系统数量激增后，企业需要跨系统复用能力并打通端到端流程。

优先保障

互操作性、可复用性、可组合性、可治理性。

相对牺牲

简单性、响应性能、轻量部署、局部演进速度。

一句话取舍

它解决了企业整合，但也让服务粒度、治理体系和 ESB 成为新的重量级约束。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



微服务

2010s

互联网业务节奏倒逼 “按业务能力拆分 + 独立部署”





这一代为什么会出现：新的约束与核心矛盾

2010s

新约束：交付频率大幅提升；核心矛盾：团队要更快迭代，但 SOA 的重治理与粗粒度服务拖慢了速度

技术条件

REST API、CI/CD、容器与 DevOps 实践逐渐成熟

业务需求

产品高频试错，团队希望围绕一块业务能力端到端负责

默认目标

按业务能力拆分，支持独立开发、独立部署、独立扩展

暂时忽略

分布式复杂度、调用链与运维成本往往被低估

这一代的关键词不是“集成”，而是“演进速度”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

2010s

上一代模式：SOA

- 很擅长企业级服务复用与流程编排
- 可以让异构系统进入统一契约体系
- 强治理适合大企业整合

新约束下的问题

- 服务边界偏粗，不易与快速变化的产品边界对齐
- 中心治理与总线容易形成变更瓶颈
- 跨团队依赖多，难以实现真正的独立发布

因此，服务必须更贴近业务能力、团队边界和发布节奏

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



微服务的结构骨架：边界、职责与协作

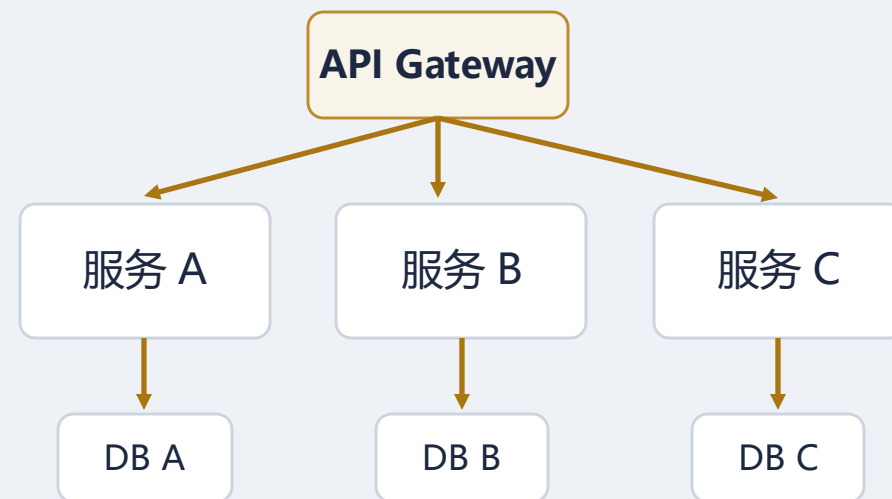
2010s

结构抓手

- 系统围绕相对独立的业务能力拆成多个小服务
- 每个服务由相对独立的团队负责其开发、部署与运维
- 接口协作替代共享实现，数据尽量跟随服务边界自治

本质：让“系统边界”“团队边界”“发布边界”尽量对齐

结构骨架





微服务如何解决核心问题

2010s

它通过更小的业务边界与独立部署，把交付链条显著缩短

1

边界更小

单次改动影响范围更可控

2

团队自治

一支团队可以端到端拥有一块能力

3

部署更独立

不必每次都整体发版整套系统

4

扩展更局部

热点能力可以单独扩容

微服务不是“服务更多”，而是“演进单位更小、交付链更短”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



微服务的质量属性取舍

2010s

微服务用“边界更小”换“交付更快”，把系统边界、团队边界与发布边界尽量对齐

核心矛盾

产品迭代太快，团队需要围绕业务能力独立开发、独立发布。

相对牺牲

一致性、运维简单性、故障定位性、全局可理解性。

优先保障

可部署性、可扩展性、可修改性、演进速度。

一句话取舍

复杂度并没有消失，而是从代码内部转移到了分布式协作、数据一致性与运维层。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

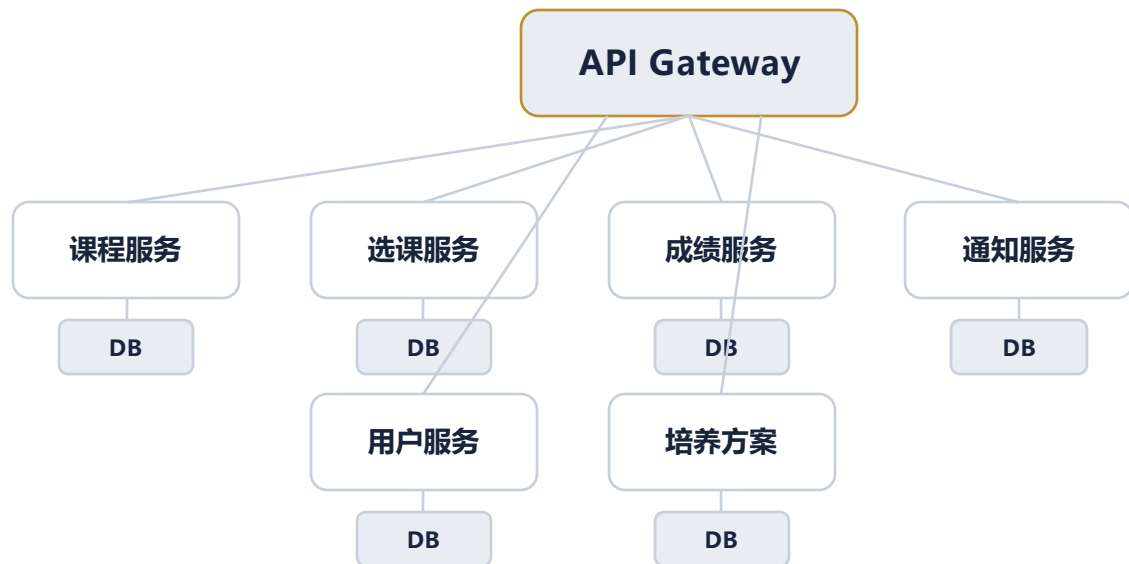
AI 增强

AI 原生



回到选课系统：微服务时代

2010s



系统边界、团队边界、发布边界开始尽量对齐。

这一代的选课系统，让“系统边界、团队边界、发布边界”尽量对齐。

1

核心变化

围绕业务能力拆成多个小服务

2

为什么这样设计

局部改动影响更小

3

新问题

调用链、数据一致性、排障更难

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

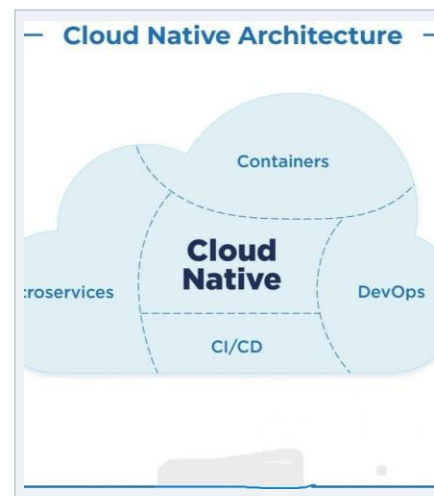
AI 原生



事件驱动 / 云原生

2015–2020s

系统不只要拆开，还要在大规模分布式环境中活得稳





这一代为什么会出现：新的约束与核心矛盾

2015–2020s

新约束：系统规模与流量波动持续放大；核心矛盾：同步链路过长、扩缩容困难、人工运维不可持续

技术条件

公有云、容器编排、流式平台、可观测性体系成熟

业务需求

系统要高弹性、自动恢复、按需扩缩容，并支持实时协同

默认目标

降低同步耦合，把运行能力下沉为平台能力

暂时忽略

端到端可理解性与时序复杂度常常会上升

复杂度开始从“开发期怎么拆”迁移到“运行期怎么活得稳”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



上一代模式为什么开始失效

2015–2020s

上一代模式：微服务

- 边界小、发布快、团队自治强
- 适合业务高频演进
- 更容易围绕业务能力扩展组织

新约束下的问题

- 全同步调用链会放大尾延迟和级联故障
- 服务变多后，发布、扩容、恢复、观测仍高度依赖人工
- 运行能力如果不平台化，规模效应就无法形成

因此，系统既要改变协作方式，也要改变运行方式

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



事件驱动 / 云原生的结构骨架：边界、职责与协作

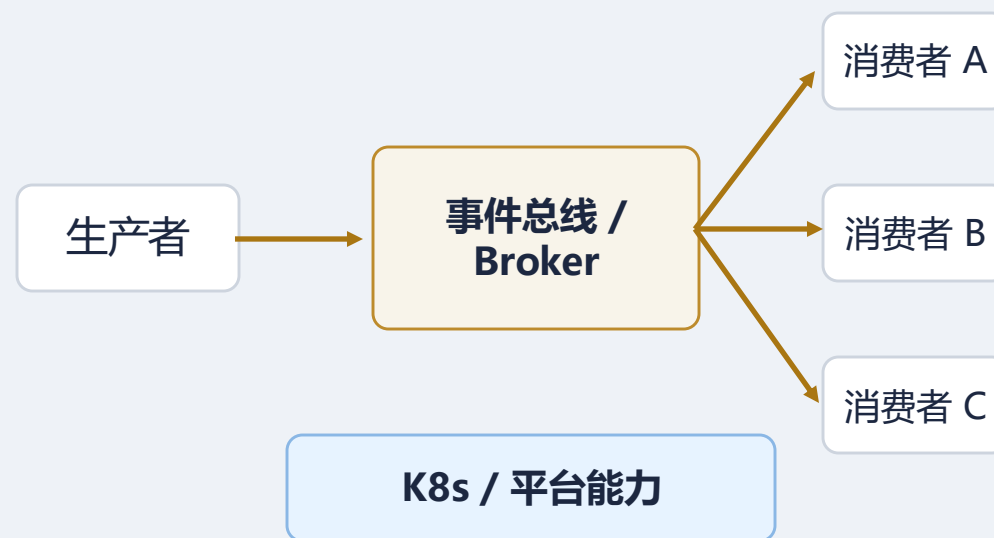
2015–2020s

结构抓手

- 事件驱动：生产者发布事实，消费者按需订阅并处理
- 消息 / 流平台承担传递、缓冲、削峰与异步解耦
- 云原生平台承担编排、扩缩容、恢复、配置与观测

本质：事件驱动解决“如何协作”，云原生解决“如何运行”

结构骨架





事件驱动 / 云原生如何解决核心问题

2015–2020s

它通过异步事件与平台化运行能力，让分布式系统更弹性、更韧性、更自动化

1

异步解耦

生产者不必强依赖消费者实时在线

2

削峰与缓冲

流量波峰不再直接压垮下游

3

自动化运行

扩容、恢复、调度尽量交给平台处理

4

可观测性前置

运行状态必须被显式监控和追踪

它把“系统运行能力”正式拉进了架构主体，而不再只是运维附属

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



事件驱动 / 云原生的质量属性取舍

2015–2020s

这一代优先解决“系统如何在大规模分布式环境中活得稳”

核心矛盾

系统规模和流量波动放大，同步链路过长、人工运维不可持续。

相对牺牲

可理解性、可调试性、时序可预测性、强一致性。

优先保障

弹性、韧性、可扩展性、可用性、自动化运维能力。

一句话取舍

它让系统更能扛、更会自恢复，但也让时序推理、排障和最终一致性治理更难。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

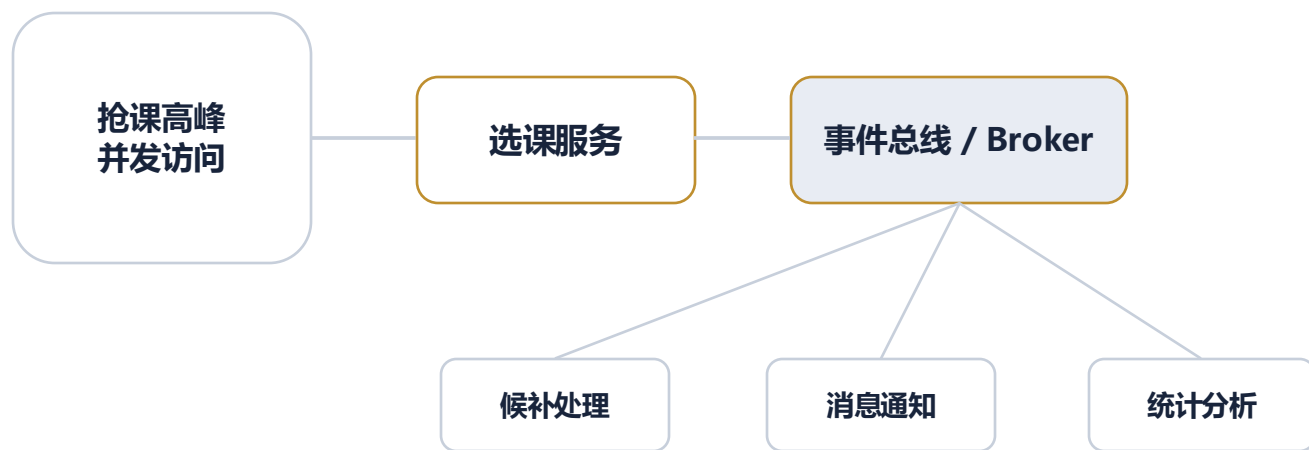
AI 增强

AI 原生



回到选课系统：事件驱动 / 云原生时代

2015–2020s



K8s 平台
扩容 / 恢复 / 观测

从“服务怎么拆”进一步走向“高峰下怎么稳定运行”。

这一代的选课系统，不只是“怎么拆”，更重要的是“怎么在高峰流量下活得稳”。

1

核心变化

从服务拆分走向异步协作与平台化运行

2

为什么这样设计

削峰填谷，高峰期自动扩缩容

3

新问题

时序更复杂，排障更依赖可观测性

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

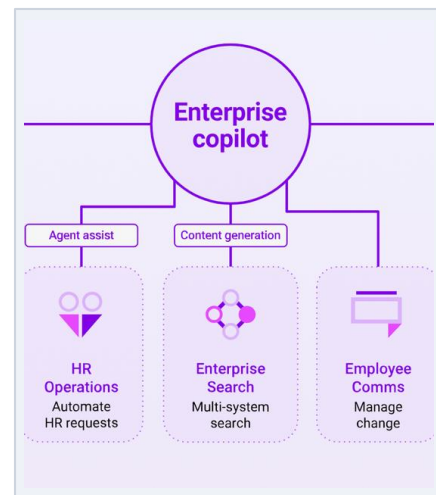
AI 原生



AI 增强

2023-至今

AI 首先以外挂能力进入既有软件，而不是立刻重写一切





这一代为什么会出现：新的约束与核心矛盾

2023-至今

新约束：用户开始直接用自然语言工作；核心矛盾：传统软件擅长规则与流程，却不擅长开放式语义任务

技术条件

LLM API、RAG、提示词编排与工具调用快速成熟

业务需求

企业希望先把 AI 接到现有系统上，而不是推翻旧系统

默认目标

给旧系统接入自然语言入口、知识增强与内容生成能力

暂时忽略

复杂长链路任务执行与非确定性治理还未成为主战场

AI 在这一阶段更像“外挂能力层”，而不是系统运行时的中心

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



AI 增强的结构骨架：边界、职责与协作

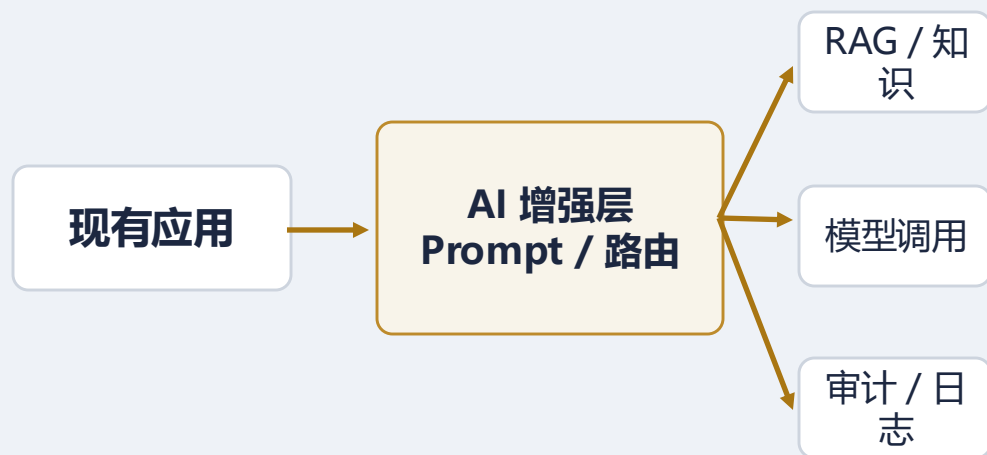
2023-至今

结构抓手

- 现有业务系统继续保留主流程与主数据
- 新增模型调用层，负责 prompt 构造、路由与上下文组织
- RAG、工具调用、审计与评估成为常见新组件

本质：传统系统继续定义主流程，AI 负责增能而不接管全流程

结构骨架



最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



AI 增强如何解决核心问题

2023-至今

它通过“旧系统保主流程 + AI 负责语义增能”，实现了低风险接入

1

自然语言入口

把复杂菜单和检索流程前移为对话入口

2

知识增强

RAG 让模型能利用企业私域知识

3

工具接入

模型不仅能回答，还能查询和触发动作

4

治理补丁

日志、权限、评估与审计开始补上

这一阶段的关键不是“重写系统”，而是“在可控边界内先验证价值”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



AI 增强的质量属性取舍

2023-至今

AI 增强的目标不是重写系统，而是以较低风险把自然语言与知识能力接到旧系统上

核心矛盾

传统系统擅长规则流程，却不擅长开放式语义理解与知识处理。

优先保障

易用性、语义适应性、知识获取效率、功能可扩展性。

相对牺牲

可预测性、可测试性、成本稳定性、安全边界清晰性。

一句话取舍

先追求“能增能、能试点”，而不是一开始就追求完全确定、完全自动化。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

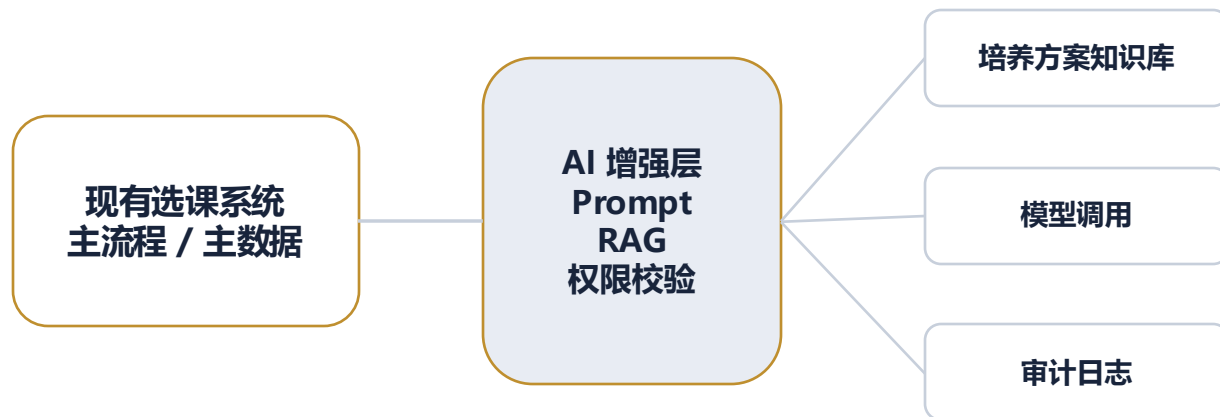
AI 增强

AI 原生



回到选课系统：AI 增强时代

2023-至今



学生提问：我还差哪些学分？

AI 负责解释、检索与生成，旧系统仍负责确定性流程。

1

核心变化

在旧系统上接入自然语言与知识能力

2

为什么这样设计

低风险验证 AI 价值

3

新问题

输出不稳定，治理和评估更难

这一代的选课系统，先让旧系统变聪明，而不是一开始就推翻重写。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

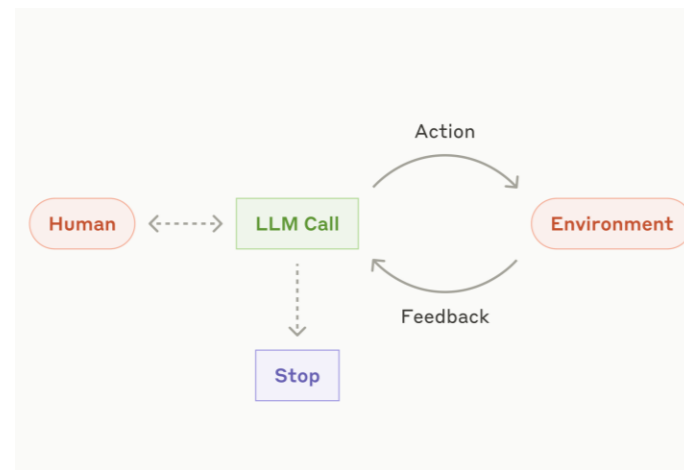
AI 原生



AI 原生

2024-至今

模型、Agent、工具、记忆与评测
开始共同构成新的系统骨架





这一代为什么会出现：新的约束与核心矛盾

2024-至今

新约束：AI 不只回答问题，还要完成任务；核心矛盾：传统请求-响应式架构无法管理长链路、非确定性与多步协作

技术条件

Agent、工具使用、长上下文、记忆与评测框架快速出现

业务需求

企业开始期待 AI 规划、执行、回顾并与人协作完成任务

默认目标

把认知流程显式工程化，让非确定性系统变得可控

暂时忽略

若盲目上多 Agent，复杂度会比价值增长得更快

真正的断点不是“有没有模型”，而是“模型是不是系统运行时的一部分”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



AI 原生的结构骨架：边界、职责与协作

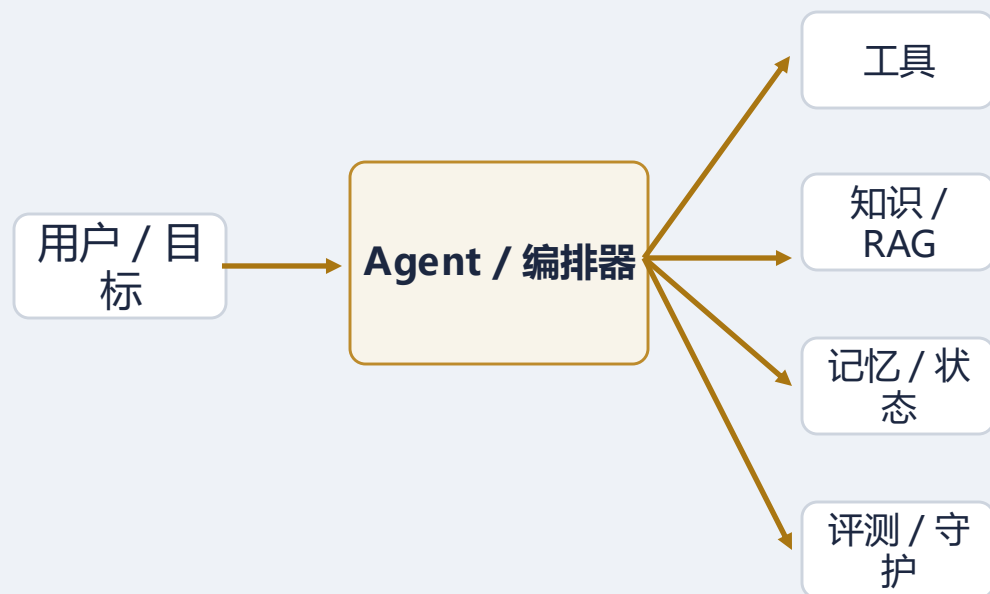
2024-至今

结构抓手

- Agent / 编排器负责目标理解、任务规划与执行流程组织
- 工具、知识、状态、记忆层成为模型可调用的运行时组件
- 评测、守护、权限与 HITL 负责把非确定性纳入治理

本质：模型不再只是外部 API，而是系统运行时主体之一

结构骨架





AI 原生如何解决核心问题

2024-至今

它通过把“上下文、工具、状态、评测、守护”显式工程化，来管理机器认知

1

显式状态

任务过程与中间结果不再隐含在单轮对话里

2

显式工具链

模型可以在边界内查询、执行、验证与回滚

3

显式评测

质量不只看可用性，还要看行为是否受控

4

显式守护

权限、成本、风险与人类接管点必须前置设计

AI 原生的真正挑战，不是“让模型更聪明”，而是“让系统更可控”

最早期

主机/终端

C/S

三层/分层

SOA

微服务

事件驱动/云原生

AI 增强

AI 原生



AI 原生的质量属性取舍

2024-至今

AI 原生系统为了获得“认知式执行能力”，必须把评测、守护、权限和 HITL 变成一等公民

核心矛盾

AI 不再只是回答问题，而要承担多步任务执行与协作。

优先保障

适应性、任务完成能力、人机协作效率、复杂 workflow 扩展能力。

相对牺牲

确定性、可验证性、成本可预测性、安全与责任边界清晰度。

一句话取舍

真正难点不是“让模型更聪明”，而是“让非确定性系统在工程上更可控”。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



回到选课系统：AI 原生时代

2024-至今



1

核心变化

从问答助手走向学业规划 Agent

2

为什么这样设计

支持多步规划、工具调用与反馈

3

新问题

权限、责任、成本和评测都更敏感

这一代的选课系统，开始从“功能系统”走向“围绕认知过程组织的任务系统”。

最早期

主机 / 终端

C/S

三层 / 分层

SOA

微服务

事件驱动 / 云原生

AI 增强

AI 原生



总结



把时代放在一起看：主要矛盾一直在迁移

最早期/主机前夜

如何高效使用昂贵算力

主机/终端

如何让很多人共享同一套核心事务系统

C/S

如何把交互能力下放给桌面用户

三层/分层

如何统一入口并把系统内职责分清

SOA

如何跨系统复用业务能力并完成企业级整合

微服务

如何让团队按业务能力独立演进与发布

事件驱动/云原生

如何在大规模分布式环境中保持韧性与自动化运行

AI 增强

如何把语义理解与知识增强接入既有软件

AI 原生

如何工程化管理机器认知、任务执行与非确定性

一句话总结：软件架构的历史，就是“在新的成本结构下，重画边界和协作方式”的历史



架构基本单元如何变化：程序 → 层 → 服务 → 事件 → 模型 / Agent

当最小治理单元变化时，测试方式、平台能力和组织边界也会跟着变化



所以：架构史不是一串名词，而是一部“系统基本单元不断变化”的历史



今天如何选型：不是追新，而是识别你的主矛盾

仍以确定性流程为主，团队规模不大

分层单体 / 模块化单体仍可能是最优解

业务域复杂、团队独立发布频繁

微服务可能合适，但必须有足够平台能力

强异步、高峰流量、实时协同

事件驱动与云原生平台优先考虑

想提升知识工作效率，但主流程仍旧确定

优先做 AI 增强，而非直接上多 Agent

任务开放、需要多步推理与工具调用

再考虑 AI 原生架构与 Agent 编排

判断顺序建议：先看需求确定性，再看组织边界，再看运行复杂度，最后才看“是不是时髦”

从“管理计算资源” 走向“管理机器认知”

如果说主机、C/S、微服务、云原生是在不断重画“计算边界”，
那么 AI 原生正在迫使我们重画“认知边界、权限边界与责任边界”





参考书籍与延伸阅读

建议按 “架构基础 → 服务化 / 分布式 → AI 系统工程” 这条线阅读

A. 架构基础

- Software Architecture in Practice (4th ed.) — Len Bass, Paul Clements, Rick Kazman, 2021
- Fundamentals of Software Architecture — Mark Richards, Neal Ford, 2020

B. 服务化 / 分布式 / 云

- Enterprise Integration Patterns — Gregor Hohpe, Bobby Woolf, 2004
- SOA: Principles of Service Design — Thomas Erl, 2007
- Building Microservices (2nd ed.) — Sam Newman, 2021
- Designing Data-Intensive Applications — Martin Kleppmann, 2017
- Cloud Native Patterns — Cornelia Davis, 2019

C. AI 系统 / AI 原生

- Designing Machine Learning Systems — Chip Huyen, 2022
- AI Engineering — Chip Huyen, 2025